

ALGORYTMY PRZETWARZANIA OBRAZÓW

Lab 3
Łukasz Roszkowiak

Operacje morfologii matematycznej

Zadanie

- Opracować algorytm i zaimplementować w aplikacji operację morfologiczną na obrazie przy użyciu wybranego elementu strukturalnego.

Operacje

- erozji,
- dylatacji,
- otwarcia,
- zamknięcia

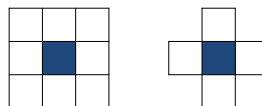
Elementy strukturalne

- romb (cztero-spójnego sąsiedztwa),
- kwadrat (ośmio-spójnego sąsiedztwa)

Op. morf. w obrazach binarnych

- Przekształcenia polegają na zmianie intensywności lub pozostawieniu intensywności punktu przykrytego przez punkt centralny elementu strukturalnego

Podstawowe elementy strukturalne



Operacje morfologiczne w OpenCV

```

- morphologyEx()
void cv::morphologyEx(InputArray src, OutputArray dst, int op, InputArray kernel)
Parameters:
src: Source image. The number of channels can be arbitrary. The depth should be one of CV_8U, CV_8S, CV_16U, CV_16S, CV_32F or CV_32F.
dst: Destination image of the same size and type as source image.
op: Type of a morphological operation, see MorphTypes.
kernel: Structuring element. It can be created using getStructuringElement().
anchor: Anchor position with the kernel. Negative values mean that the anchor is at the kernel center.
iterations: Number of times erosion and dilation are applied.
borderType: Border interpolation method, see BorderTypes. BORDER_DEFAULT is not supported.
borderValue: Border value in case of a constant border. The default value has a special meaning.
Python:
cv.morphologyEx(src, dst, op, kernel, iterations, borderType, borderValue)

```

Typy op. morf. w OpenCV

MorphTypes	
enum cv::MorphTypes	
Morphological operation	
Type of morphological operation	
cv::MORPH_ERODE	erode
cv::MORPH_DILATE	dilate
cv::MORPH_OPEN	opening operation
cv::MORPH_CLOSE	closing operation
cv::MORPH_GRAB	grab operation
cv::MORPH_HITTEST	hit test
cv::MORPH_HITTEST_EX	hit test with extended kernel
cv::MORPH_HITTEST_EX2	hit test with extended kernel 2
cv::MORPH_HITTEST_EX3	hit test with extended kernel 3
cv::MORPH_HITTEST_EX4	hit test with extended kernel 4
cv::MORPH_HITTEST_EX5	hit test with extended kernel 5
cv::MORPH_HITTEST_EX6	hit test with extended kernel 6
cv::MORPH_HITTEST_EX7	hit test with extended kernel 7
cv::MORPH_HITTEST_EX8	hit test with extended kernel 8
cv::MORPH_HITTEST_EX9	hit test with extended kernel 9
cv::MORPH_HITTEST_EX10	hit test with extended kernel 10
cv::MORPH_HITTEST_EX11	hit test with extended kernel 11
cv::MORPH_HITTEST_EX12	hit test with extended kernel 12
cv::MORPH_HITTEST_EX13	hit test with extended kernel 13
cv::MORPH_HITTEST_EX14	hit test with extended kernel 14
cv::MORPH_HITTEST_EX15	hit test with extended kernel 15
cv::MORPH_HITTEST_EX16	hit test with extended kernel 16
cv::MORPH_HITTEST_EX17	hit test with extended kernel 17
cv::MORPH_HITTEST_EX18	hit test with extended kernel 18
cv::MORPH_HITTEST_EX19	hit test with extended kernel 19
cv::MORPH_HITTEST_EX20	hit test with extended kernel 20
cv::MORPH_HITTEST_EX21	hit test with extended kernel 21
cv::MORPH_HITTEST_EX22	hit test with extended kernel 22
cv::MORPH_HITTEST_EX23	hit test with extended kernel 23
cv::MORPH_HITTEST_EX24	hit test with extended kernel 24
cv::MORPH_HITTEST_EX25	hit test with extended kernel 25
cv::MORPH_HITTEST_EX26	hit test with extended kernel 26
cv::MORPH_HITTEST_EX27	hit test with extended kernel 27
cv::MORPH_HITTEST_EX28	hit test with extended kernel 28
cv::MORPH_HITTEST_EX29	hit test with extended kernel 29
cv::MORPH_HITTEST_EX30	hit test with extended kernel 30
cv::MORPH_HITTEST_EX31	hit test with extended kernel 31
cv::MORPH_HITTEST_EX32	hit test with extended kernel 32
cv::MORPH_HITTEST_EX33	hit test with extended kernel 33
cv::MORPH_HITTEST_EX34	hit test with extended kernel 34
cv::MORPH_HITTEST_EX35	hit test with extended kernel 35
cv::MORPH_HITTEST_EX36	hit test with extended kernel 36
cv::MORPH_HITTEST_EX37	hit test with extended kernel 37
cv::MORPH_HITTEST_EX38	hit test with extended kernel 38
cv::MORPH_HITTEST_EX39	hit test with extended kernel 39
cv::MORPH_HITTEST_EX40	hit test with extended kernel 40
cv::MORPH_HITTEST_EX41	hit test with extended kernel 41
cv::MORPH_HITTEST_EX42	hit test with extended kernel 42
cv::MORPH_HITTEST_EX43	hit test with extended kernel 43
cv::MORPH_HITTEST_EX44	hit test with extended kernel 44
cv::MORPH_HITTEST_EX45	hit test with extended kernel 45
cv::MORPH_HITTEST_EX46	hit test with extended kernel 46
cv::MORPH_HITTEST_EX47	hit test with extended kernel 47
cv::MORPH_HITTEST_EX48	hit test with extended kernel 48
cv::MORPH_HITTEST_EX49	hit test with extended kernel 49
cv::MORPH_HITTEST_EX50	hit test with extended kernel 50
cv::MORPH_HITTEST_EX51	hit test with extended kernel 51
cv::MORPH_HITTEST_EX52	hit test with extended kernel 52
cv::MORPH_HITTEST_EX53	hit test with extended kernel 53
cv::MORPH_HITTEST_EX54	hit test with extended kernel 54
cv::MORPH_HITTEST_EX55	hit test with extended kernel 55
cv::MORPH_HITTEST_EX56	hit test with extended kernel 56
cv::MORPH_HITTEST_EX57	hit test with extended kernel 57
cv::MORPH_HITTEST_EX58	hit test with extended kernel 58
cv::MORPH_HITTEST_EX59	hit test with extended kernel 59
cv::MORPH_HITTEST_EX60	hit test with extended kernel 60
cv::MORPH_HITTEST_EX61	hit test with extended kernel 61
cv::MORPH_HITTEST_EX62	hit test with extended kernel 62
cv::MORPH_HITTEST_EX63	hit test with extended kernel 63
cv::MORPH_HITTEST_EX64	hit test with extended kernel 64
cv::MORPH_HITTEST_EX65	hit test with extended kernel 65
cv::MORPH_HITTEST_EX66	hit test with extended kernel 66
cv::MORPH_HITTEST_EX67	hit test with extended kernel 67
cv::MORPH_HITTEST_EX68	hit test with extended kernel 68
cv::MORPH_HITTEST_EX69	hit test with extended kernel 69
cv::MORPH_HITTEST_EX70	hit test with extended kernel 70
cv::MORPH_HITTEST_EX71	hit test with extended kernel 71
cv::MORPH_HITTEST_EX72	hit test with extended kernel 72
cv::MORPH_HITTEST_EX73	hit test with extended kernel 73
cv::MORPH_HITTEST_EX74	hit test with extended kernel 74
cv::MORPH_HITTEST_EX75	hit test with extended kernel 75
cv::MORPH_HITTEST_EX76	hit test with extended kernel 76
cv::MORPH_HITTEST_EX77	hit test with extended kernel 77
cv::MORPH_HITTEST_EX78	hit test with extended kernel 78
cv::MORPH_HITTEST_EX79	hit test with extended kernel 79
cv::MORPH_HITTEST_EX80	hit test with extended kernel 80
cv::MORPH_HITTEST_EX81	hit test with extended kernel 81
cv::MORPH_HITTEST_EX82	hit test with extended kernel 82
cv::MORPH_HITTEST_EX83	hit test with extended kernel 83
cv::MORPH_HITTEST_EX84	hit test with extended kernel 84
cv::MORPH_HITTEST_EX85	hit test with extended kernel 85
cv::MORPH_HITTEST_EX86	hit test with extended kernel 86
cv::MORPH_HITTEST_EX87	hit test with extended kernel 87
cv::MORPH_HITTEST_EX88	hit test with extended kernel 88
cv::MORPH_HITTEST_EX89	hit test with extended kernel 89
cv::MORPH_HITTEST_EX90	hit test with extended kernel 90
cv::MORPH_HITTEST_EX91	hit test with extended kernel 91
cv::MORPH_HITTEST_EX92	hit test with extended kernel 92
cv::MORPH_HITTEST_EX93	hit test with extended kernel 93
cv::MORPH_HITTEST_EX94	hit test with extended kernel 94
cv::MORPH_HITTEST_EX95	hit test with extended kernel 95
cv::MORPH_HITTEST_EX96	hit test with extended kernel 96
cv::MORPH_HITTEST_EX97	hit test with extended kernel 97
cv::MORPH_HITTEST_EX98	hit test with extended kernel 98
cv::MORPH_HITTEST_EX99	hit test with extended kernel 99

Elementy strukturalne w OpenCV

• MorphShapes	
enum cv::MorphShapes	
#include <opencv2/opencv.hpp>	
shape of the structuring element	
Enumerator	
MORPH_RECT Python: cv.MORPH_RECT	a rectangular structuring element. $E_{ij} = 1$
MORPH_CROSS Python: cv.MORPH_CROSS	a cross-shaped structuring element. $E_{ij} = \begin{cases} 1 & \text{if } i = \text{anchor.y or } j = \text{anchor.x} \\ 0 & \text{otherwise} \end{cases}$
MORPH_ELLIPSE Python: cv.MORPH_ELLIPSE	an elliptic structuring element, that is, a filled ellipse inscribed into the rectangle Rect(0, 0, size.width, 0, size.height)

Skrajne piksele

Różne wersje uzupełniania pikseli granicy obrazu:

- Stała wartość (constant) iiiiii | abcdef | iiiiii
- Powielenie (replicate) aaaa | abcdef | eeee
- Odbicie (reflect) dcba | abcdef | fedc
- Odbicie v2 (reflect_101) edcb | abcdef | edcb
- Zawinięcie (wrap) cdef | abcdef | abcd

Operacja szkieletyzacji

Zadanie

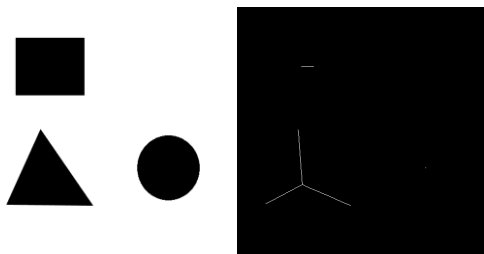
Opracować algorytm i zaimplementować w aplikacji realizację algorytmu ścieniania

Przekształcenie obiektu w jego szkielet czyli zbiór punktów równoodległych od co najmniej dwóch krawędzi obiektu.

APO

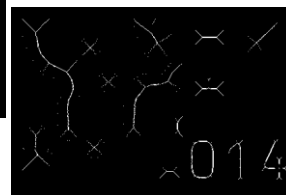


Szkielety podstawowych kształtów



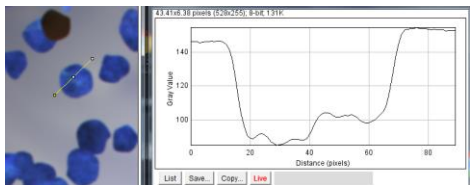
Szkieletyzacja (przykład)

- znalezienie szkieletu czyli punktów obiektu równoodległych od jego brzegów



Zadanie

Opracować algorytm i uruchomić aplikację realizującą wizualizację wartości obrazu wzdłuż zadanej linii prostej ograniczonej dwoma punktami – tzw. linia profilu.

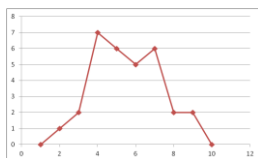


Linia profilu

0	0	0	0	0	0	0	0	0	0
0	2	2	2	1	2	2	2	1	0
0	2	2	2	1	1	2	2	4	0
0	1	2	2	7	6	6	2	1	0
0	1	2	7	6	5	6	2	2	0
0	1	7	6	5	5	6	1	2	0
0	1	7	7	6	6	6	2	2	0
0	1	2	1	1	2	2	1	5	0
0	1	2	1	1	1	2	2	3	0
0	0	0	0	0	0	0	0	0	0

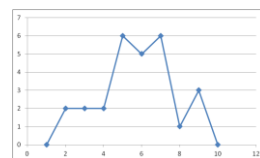
Linia profilu

0	0	0	0	0	0	0	0	0	0
0	2	2	2	1	2	2	2	1	0
0	2	2	2	1	1	2	2	1	0
0	1	2	2	7	6	6	2	1	0
0	1	2	7	6	5	6	2	2	0
0	1	7	6	5	5	6	1	2	0
0	1	7	7	6	6	6	2	2	0
0	1	2	1	1	2	2	1	5	0
0	1	2	1	1	1	2	2	3	0
0	0	0	0	0	0	0	0	0	0



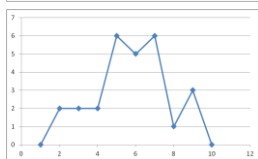
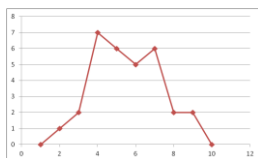
Linia profilu

0	0	0	0	0	0	0	0	0	0
0	2	2	2	1	2	2	2	1	0
0	2	2	2	1	1	2	2	1	0
0	1	2	2	7	6	6	2	1	0
0	1	2	7	6	5	6	2	2	0
0	1	7	6	5	5	6	1	2	0
0	1	7	7	6	6	6	2	2	0
0	1	2	1	1	2	2	1	5	0
0	1	2	1	1	1	2	2	3	0
0	0	0	0	0	0	0	0	0	0

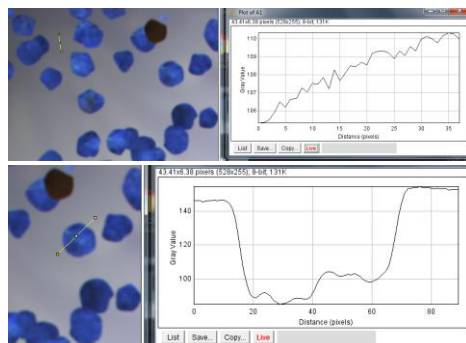


Linia profilu

0	0	0	0	0	0	0	0	0	0
0	2	2	2	1	2	2	2	1	0
0	2	2	2	1	1	2	2	1	0
0	1	2	2	7	6	6	2	1	0
0	1	2	7	6	5	6	2	2	0
0	1	7	6	5	5	6	1	2	0
0	1	7	7	6	6	6	2	2	0
0	1	2	1	1	2	2	1	5	0
0	1	2	1	1	1	2	2	3	0
0	0	0	0	0	0	0	0	0	0

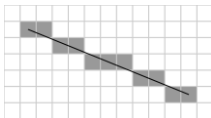


Linia profilu (2)



Jak znaleźć wartości linii profilu?

- Bresenham's Line Generation Algorithm



- Przykład:

<https://www.geeksforgeeks.org/bresenham-line-generation-algorithm/>

```
def bresenham_line(x0, y0, x1, y1):
    """Bresenham's Line Algorithm"""
    points = []
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    sx = x0 < x1
    sy = y0 < y1
    dx = abs(dx)
    dy = abs(dy)
    if dx > dy:
        stepx = 1 if sx else -1
        stepy = 0
        for i in range(dx):
            x = x0 + stepx * i
            y = y0
            points.append((x, y))
    else:
        stepx = 0
        stepy = 1 if sy else -1
        for i in range(dy):
            x = x0
            y = y0 + stepy * i
            points.append((x, y))
    return points
```

Piramida obrazów

Zadanie

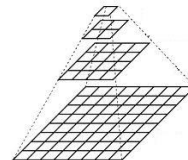
Opracować algorytm i uruchomić aplikację realizującą piramidę obrazów ograniczoną do 2 rozmiarów w górę i w dół



*źródło obrazów: wikipedia.org

Piramida obrazów

- to zbiór obrazów powstałych z jednego oryginalnego obrazu
- sukcesywnie próbkowane w dół (lub w górę), aż do osiągnięcia pożądanego punktu zatrzymania



Rodzaje piramid

Istnieją dwa popularne rodzaje piramid obrazu:

- Piramida gaussowska:**
 - Używana do próbkowania w dół obrazów
 - `dst = cv.pyrDown(src, dstsize=(cols // 2, rows // 2))`
- Piramida Laplaciana:**
 - Służy do rekonstrukcji obrazu o wyższej rozdzielczości z obrazu znajdującego się niżej w piramidzie (o niższej rozdzielczości)
 - `dst = cv.pyrUp(src, dstsize=(2 * cols, 2 * rows))`

Zalety piramid

- Proste i szybkie rozwiązanie
- Często wykorzystywane jako opcja „zoom”

• pyrUp()

```
void cv_pyrUp ( InputArray src,
                OutputArray dst,
                const Size & dstsize = Size(),
                int borderType = BORDER_DEFAULT )
```

Python:
`cv.pyrUp(src, dst, dstsize, borderType) -> dst`

• pyrDown()

```
void cv_pyrDown ( InputArray src,
                  OutputArray dst,
                  const Size & dstsize = Size(),
                  int borderType = BORDER_DEFAULT )
```

Python:
`cv.pyrDown(src, dst, dstsize, borderType) -> dst`

Podsumowanie Lab3

- Operacje morfologii matematycznej
- Szykieletyzacja
- Detekcja prostych
- Linia profilu
- Piramida obrazów